SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Industry Problem Solving Task)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO4-AT3-INDUSTRY PROBLEM SOLVING TASK
Question	Q1 — Problem 1: Smart Water Billing System (50 Marks)

Problem Context

A municipal corporation needs a system to calculate water bills for Residential, Commercial, and Industrial users with different tariff rates and slab structures. The system reads customer data from a CSV file (customer ID, type, consumption in kilolitres). Slab-based rates apply per category, Commercial and Industrial slabs differ from Residential, a 5% late fee is applied when the payment date is past due, all tariff rules reside in a JSON configuration file loaded at runtime, and the system generates a formatted bill report in a text file while remaining easy to extend with new customer categories.

Task (a) — WaterConsumer base class and subclasses [10 M]

An abstract base class WaterConsumer is defined using Python's abc module, with an abstract method compute_bill() that every subclass must implement. It validates that consumption is not negative in the constructor (raising NegativeConsumptionError otherwise) and provides a shared _slab_amount() helper that charges consumption progressively across slab boundaries — this is reused by all three subclasses since only the slab boundaries and rates differ between categories, which is exactly what makes the design easy to extend. ResidentialConsumer, CommercialConsumer, and IndustrialConsumer each override compute_bill() to apply their category's slab rules, loaded at runtime from the JSON configuration.

consumers.py

```
"""
consumers.py
------------------------------------------------------------
Task (a): Base class WaterConsumer with an abstract method
compute_bill(). Derived classes ResidentialConsumer,
CommercialConsumer, and IndustrialConsumer override
compute_bill() to apply their own slab pricing logic.
"""

from abc import ABC, abstractmethod
from exceptions import NegativeConsumptionError


class WaterConsumer(ABC):
    """Abstract base class for all water consumer categories."""

    def __init__(self, customer_id, consumption_kl, slabs):
        if consumption_kl < 0:
            raise NegativeConsumptionError(consumption_kl, customer_id)
        self.customer_id = customer_id
        self.consumption_kl = consumption_kl
```

```
        self.slabs = slabs  # list of {"upto": int|None, "rate": float}

    @abstractmethod
    def compute_bill(self):
        """Return the base water bill (before any late fee) as a float."""
        raise NotImplementedError

    def _slab_amount(self):
        """
        Shared slab-billing logic: charges consumption progressively across
        slab boundaries, e.g. 0-20 kL at rate1, 21-50 kL at rate2, above 50
        kL at rate3. Reused by every subclass since only the slab
        boundaries/rates differ per category.
        """
        remaining = self.consumption_kl
        lower_bound = 0
        total = 0.0
        for slab in self.slabs:
            upto = slab["upto"]
            rate = slab["rate"]
            if upto is None:
                slab_units = remaining
            else:
                slab_units = max(0, min(remaining, upto - lower_bound))
            total += slab_units * rate
            remaining -= slab_units
            if upto is not None:
                lower_bound = upto
            if remaining <= 0:
                break
        return total


class ResidentialConsumer(WaterConsumer):
    """Residential category: standard slab pricing."""

    def compute_bill(self):
        return self._slab_amount()


class CommercialConsumer(WaterConsumer):
    """Commercial category: higher slab pricing than Residential."""

    def compute_bill(self):
        return self._slab_amount()


class IndustrialConsumer(WaterConsumer):
    """Industrial category: highest slab pricing."""

    def compute_bill(self):
        return self._slab_amount()


# Maps a category name from customers.csv to its consumer class.
CATEGORY_CLASS_MAP = {
    "Residential": ResidentialConsumer,
    "Commercial": CommercialConsumer,
    "Industrial": IndustrialConsumer,
}
```

Task (b) — tariff_loader module and tariff_config.json [10 M]

The tariff_loader module reads the JSON tariff configuration file and returns a structured dictionary mapping each category name to its late fee percentage and its list of slab rules ({"upto": kL_limit, "rate": rate_per_kL}), where the final slab's upto is null to represent 'above the last limit'. It validates the structure while loading and raises a custom

MalformedConfigError if the file is missing, not valid JSON, or missing required keys.

tariff_config.json

```
{
    "Residential": {
        "late_fee_percent": 5,
        "slabs": [
            {"upto": 20, "rate": 20},
            {"upto": 50, "rate": 35},
            {"upto": null, "rate": 50}
        ]
    },
    "Commercial": {
        "late_fee_percent": 5,
        "slabs": [
            {"upto": 20, "rate": 30},
            {"upto": 50, "rate": 45},
            {"upto": null, "rate": 65}
        ]
    },
    "Industrial": {
        "late_fee_percent": 5,
        "slabs": [
            {"upto": 20, "rate": 40},
            {"upto": 50, "rate": 60},
            {"upto": null, "rate": 85}
        ]
    }
}
```

tariff_loader.py

```
"""
tariff_loader.py
----------------------------------------------------------------
Task (b): Reads the JSON tariff configuration file and returns
a structured dictionary mapping category -> slab rules.
"""

import json


class MalformedConfigError(Exception):
    """Raised when tariff_config.json is missing required structure."""
    pass


def load_tariff_config(path="tariff_config.json"):
    """
    Reads a JSON tariff configuration file and returns a dictionary of the
    form:
        {
            "Residential": {
                "late_fee_percent": 5,
                "slabs": [{"upto": 20, "rate": 20}, ...]
            },
            "Commercial": {...},
            "Industrial": {...}
        }

    Raises:
        MalformedConfigError: if the file is not valid JSON, or a category
            entry is missing "slabs" / "late_fee_percent", or a slab entry
            is missing "rate".
    """
    try:
        with open(path, "r") as f:
            raw = json.load(f)
    except FileNotFoundError:
```

```
        raise MalformedConfigError(f"Tariff config file not found: {path}")
    except json.JSONDecodeError as e:
        raise MalformedConfigError(f"Tariff config is not valid JSON: {e}")

    if not isinstance(raw, dict) or not raw:
        raise MalformedConfigError("Tariff config must be a non-empty JSON object")

    structured = {}
    for category, rules in raw.items():
        if "slabs" not in rules or "late_fee_percent" not in rules:
            raise MalformedConfigError(
                f"Category '{category}' is missing 'slabs' or 'late_fee_percent'"
            )
        for slab in rules["slabs"]:
            if "rate" not in slab:
                raise MalformedConfigError(
                    f"A slab in category '{category}' is missing 'rate'"
                )
        structured[category] = rules

    return structured


if __name__ == "__main__":
    config = load_tariff_config("tariff_config.json")
    for category, rules in config.items():
        print(f"{category}: late fee {rules['late_fee_percent']}%, "
              f"{len(rules['slabs'])} slabs -> {rules['slabs']}")
```

Task (c) — Core billing engine [12 M]

billing_engine.py reads customers.csv row by row, validates each record, instantiates the correct WaterConsumer subclass via the CATEGORY_CLASS_MAP, applies the loaded slab rates to compute the base bill, and compares payment_date against due_date to add a 5% late fee whenever payment was made after the due date. Every computed bill is written to bills_report.txt.

customers.csv (sample input)

```
customer_id,category,consumption_kl,due_date,payment_date
C001,Residential,15,2026-07-10,2026-07-08
C002,Residential,35,2026-07-10,2026-07-15
C003,Residential,65,2026-07-10,2026-07-09
C004,Commercial,18,2026-07-10,2026-07-20
C005,Commercial,45,2026-07-10,2026-07-05
C006,Commercial,80,2026-07-10,2026-07-10
C007,Industrial,10,2026-07-10,2026-07-01
C008,Industrial,40,2026-07-10,2026-07-18
C009,Industrial,120,2026-07-10,2026-07-11
C010,Residential,20,2026-07-10,2026-07-09
```

billing_engine.py

```
"""
billing_engine.py
------------------------------------------------------------
Task (c): Core billing engine. Reads customers.csv, instantiates
the correct consumer class per record, applies the loaded slab
rates, adds a 5% late fee when payment is past due, and writes
results to bills_report.txt.

Task (d): Validates every input row and raises/catches custom
exceptions for missing fields, negative consumption, unrecognised
category, and malformed JSON config.

Task (e): Also produces summary_report.txt with total revenue
per category.
"""

import csv
from datetime import datetime
from collections import defaultdict

from tariff_loader import load_tariff_config, MalformedConfigError
from consumers import CATEGORY_CLASS_MAP
from exceptions import InvalidCategoryError, NegativeConsumptionError, MissingFieldError

REQUIRED_FIELDS = ["customer_id", "category", "consumption_kl", "due_date", "payment_date"]
DATE_FORMAT = "%Y-%m-%d"


def validate_row(row):
    """
    Task (d): Validates a single CSV row.
    Raises MissingFieldError, InvalidCategoryError, or NegativeConsumptionError
    as appropriate. Returns the row with consumption_kl converted to float.
    """
    for field in REQUIRED_FIELDS:
        if row.get(field) is None or str(row.get(field)).strip() == "":
            raise MissingFieldError(field, row.get("customer_id"))

    if row["category"] not in CATEGORY_CLASS_MAP:
        raise InvalidCategoryError(row["category"], row["customer_id"])

    try:
        consumption = float(row["consumption_kl"])
    except ValueError:
        raise MissingFieldError("consumption_kl (non-numeric)", row["customer_id"])
```

```
    if consumption < 0:
        raise NegativeConsumptionError(consumption, row["customer_id"])

    row["consumption_kl"] = consumption
    return row


def process_customers(csv_path, tariff_config):
    """
    Reads customers.csv, validates each row, computes each customer's bill
    (with late fee where applicable), and returns a list of result dicts
    plus a list of (row, error) pairs for any rows that failed validation.
    """
    results = []
    errors = []

    with open(csv_path, newline="") as f:
        reader = csv.DictReader(f)
        for raw_row in reader:
            try:
                row = validate_row(dict(raw_row))
            except (MissingFieldError, InvalidCategoryError, NegativeConsumptionError) as e:
                errors.append((raw_row, e))
                continue

            category = row["category"]
            slabs = tariff_config[category]["slabs"]
            late_fee_percent = tariff_config[category]["late_fee_percent"]

            consumer_cls = CATEGORY_CLASS_MAP[category]
            consumer = consumer_cls(row["customer_id"], row["consumption_kl"], slabs)
            base_bill = consumer.compute_bill()

            due_date = datetime.strptime(row["due_date"], DATE_FORMAT)
            payment_date = datetime.strptime(row["payment_date"], DATE_FORMAT)
            is_late = payment_date > due_date

            late_fee = round(base_bill * late_fee_percent / 100, 2) if is_late else 0.0
            total_due = round(base_bill + late_fee, 2)

            results.append({
                "customer_id": row["customer_id"],
                "category": category,
                "consumption_kl": row["consumption_kl"],
                "base_bill": round(base_bill, 2),
                "late_fee": late_fee,
                "total_due": total_due,
                "is_late": is_late,
            })

    return results, errors


def write_bills_report(results, errors, out_path="bills_report.txt"):
    """Task (e): Writes a formatted bill report for every successfully billed customer."""
    with open(out_path, "w") as f:
        f.write("SMART WATER BILLING SYSTEM - BILLS REPORT\n")
        f.write("=" * 60 + "\n")
        f.write(f"{'ID':<8}{'Category':<13}{'Consumption':<13}{'Base Bill':<12}"
                f"{'Late Fee':<11}{'Total Due':<10}\n")
        f.write("-" * 60 + "\n")
        for r in results:
            f.write(
                f"{r['customer_id']:<8}{r['category']:<13}"
                f"{r['consumption_kl']:<13.1f}Rs.{r['base_bill']:<9.2f}"
                f"Rs.{r['late_fee']:<8.2f}Rs.{r['total_due']:<7.2f}\n"
            )
        f.write("-" * 60 + "\n")
```

```
        f.write(f"Total customers billed: {len(results)}\n")

        if errors:
            f.write("\nREJECTED RECORDS (validation errors)\n")
            f.write("-" * 60 + "\n")
            for raw_row, err in errors:
                cid = raw_row.get("customer_id", "UNKNOWN")
                f.write(f"{cid}: {type(err).__name__} - {err}\n")


def write_summary_report(results, out_path="summary_report.txt"):
    """Task (e): Writes total revenue per category."""
    totals = defaultdict(float)
    counts = defaultdict(int)
    for r in results:
        totals[r["category"]] += r["total_due"]
        counts[r["category"]] += 1

    with open(out_path, "w") as f:
        f.write("SMART WATER BILLING SYSTEM - SUMMARY REPORT\n")
        f.write("=" * 45 + "\n")
        f.write(f"{'Category':<15}{'Customers':<12}{'Total Revenue':<15}\n")
        f.write("-" * 45 + "\n")
        grand_total = 0.0
        for category in sorted(totals):
            f.write(f"{category:<15}{counts[category]:<12}Rs.{totals[category]:<12.2f}\n")
            grand_total += totals[category]
        f.write("-" * 45 + "\n")
        f.write(f"{'GRAND TOTAL':<27}Rs.{grand_total:<12.2f}\n")


if __name__ == "__main__":
    try:
        config = load_tariff_config("tariff_config.json")
    except MalformedConfigError as e:
        print(f"Fatal: could not load tariff config - {e}")
        raise SystemExit(1)

    results, errors = process_customers("customers.csv", config)
    write_bills_report(results, errors, "bills_report.txt")
    write_summary_report(results, "summary_report.txt")

    print(f"Processed {len(results)} customers successfully, {len(errors)} rejected.")
    print("Reports written: bills_report.txt, summary_report.txt")
```

Task (d) — Input validation and custom exceptions [8 M]

Every customer row is validated before billing: missing/blank required fields raise MissingFieldError, a negative consumption value raises NegativeConsumptionError (also enforced again inside the WaterConsumer constructor as a second line of defence), and an unrecognised category (one not present in CATEGORY_CLASS_MAP) raises InvalidCategoryError. A malformed tariff_config.json is caught as MalformedConfigError by tariff_loader itself. All four custom exceptions derive from a common BillingError base class. process_customers() catches each error per-row so that one bad record does not stop the entire billing run — it is collected and reported separately instead.

exceptions.py

```
"""
exceptions.py
------------------------------------------------------------
Task (d): Custom exceptions for input validation.
"""


class BillingError(Exception):
    """Base class for all billing-related errors."""
    pass


class InvalidCategoryError(BillingError):
    """Raised when a customer's category is not a recognised tariff category."""
    def __init__(self, category, customer_id=None):
        self.category = category
        self.customer_id = customer_id
        msg = f"Unrecognised customer category '{category}'"
        if customer_id:
            msg += f" (customer {customer_id})"
        super().__init__(msg)


class NegativeConsumptionError(BillingError):
    """Raised when consumption_kl is negative."""
    def __init__(self, value, customer_id=None):
        self.value = value
        self.customer_id = customer_id
        msg = f"Negative consumption value: {value} kL"
        if customer_id:
            msg += f" (customer {customer_id})"
        super().__init__(msg)


class MissingFieldError(BillingError):
    """Raised when a required customer field is missing or blank."""
    def __init__(self, field, customer_id=None):
        self.field = field
        self.customer_id = customer_id
        msg = f"Missing required field '{field}'"
        if customer_id:
            msg += f" (customer {customer_id})"
        super().__init__(msg)
```

Validation test: bad_customers.csv and a malformed JSON config

bad_customers.csv

```
customer_id,category,consumption_kl,due_date,payment_date
C101,Residential,,2026-07-10,2026-07-09
C102,Commercial,-12,2026-07-10,2026-07-09
C103,Agricultural,25,2026-07-10,2026-07-09
```

test_validation.py

```
"""
test_validation.py
------------------------------------------------------------
```

```
Task (d): Demonstrates validation of bad_customers.csv (missing
field, negative consumption, unrecognised category) and a
malformed JSON tariff config.
"""

from tariff_loader import load_tariff_config, MalformedConfigError
from billing_engine import process_customers

config = load_tariff_config("tariff_config.json")

print("--- Validating bad_customers.csv ---")
results, errors = process_customers("bad_customers.csv", config)
print(f"Accepted: {len(results)}, Rejected: {len(errors)}")
for raw_row, err in errors:
    print(f"  {raw_row.get('customer_id')}: {type(err).__name__} -> {err}")

print("\n--- Loading a malformed JSON config ---")
with open("bad_tariff_config.json", "w") as f:
    f.write('{ "Residential": { "slabs": [ this is not valid json ] }')

try:
    load_tariff_config("bad_tariff_config.json")
except MalformedConfigError as e:
    print(f"Caught MalformedConfigError: {e}")
```

Actual output of test_validation.py

```
--- Validating bad_customers.csv ---
Accepted: 0, Rejected: 3
  C101: MissingFieldError -> Missing required field 'consumption_kl' (customer C101)
  C102: NegativeConsumptionError -> Negative consumption value: -12.0 kL (customer C102)
  C103: InvalidCategoryError -> Unrecognised customer category 'Agricultural' (customer C103)

--- Loading a malformed JSON config ---
Caught MalformedConfigError: Tariff config is not valid JSON: Expecting value: line 1 column 31 (cha
r 30)
```

Task (e) — bills_report.txt and summary_report.txt [10 M]

Running billing_engine.py on the 10 valid records in customers.csv produces a consolidated bills_report.txt listing each customer's ID, category, consumption, computed base bill, late fee (0 where payment was on time), and total amount due, followed by a summary_report.txt showing total revenue per category. Both were generated by actually running the program end-to-end; the exact console confirmation and file contents are reproduced below.

Console output of: python3 billing_engine.py

```
Processed 10 customers successfully, 0 rejected.
Reports written: bills_report.txt, summary_report.txt
```

bills_report.txt (actual generated content)

```
SMART WATER BILLING SYSTEM - BILLS REPORT
============================================================
ID      Category     Consumption  Base Bill   Late Fee   Total Due
------------------------------------------------------------
C001    Residential  15.0         Rs.300.00   Rs.0.00    Rs.300.00
C002    Residential  35.0         Rs.925.00   Rs.46.25   Rs.971.25
C003    Residential  65.0         Rs.2200.00  Rs.0.00    Rs.2200.00
C004    Commercial   18.0         Rs.540.00   Rs.27.00   Rs.567.00
C005    Commercial   45.0         Rs.1725.00  Rs.0.00    Rs.1725.00
C006    Commercial   80.0         Rs.3900.00  Rs.0.00    Rs.3900.00
C007    Industrial   10.0         Rs.400.00   Rs.0.00    Rs.400.00
C008    Industrial   40.0         Rs.2000.00  Rs.100.00  Rs.2100.00
C009    Industrial   120.0        Rs.8550.00  Rs.427.50  Rs.8977.50
C010    Residential  20.0         Rs.400.00   Rs.0.00    Rs.400.00
------------------------------------------------------------
Total customers billed: 10
```

summary_report.txt (actual generated content)

```
SMART WATER BILLING SYSTEM - SUMMARY REPORT
============================================
Category       Customers   Total Revenue
--------------------------------------------
Commercial     3           Rs.6192.00
Industrial     3           Rs.11477.50
Residential    4           Rs.3871.25
--------------------------------------------
GRAND TOTAL                Rs.21540.75
```

Sample verification: Customer C002 (Residential, 35 kL) is billed 20 kL × Rs.20 + 15 kL × Rs.35 = Rs.400 + Rs.525 = Rs.925 base, and since the payment date (2026-07-15) is after the due date (2026-07-10), a 5% late fee of Rs.46.25 is added for a total of Rs.971.25 — exactly matching the report above.

Task (f) — System extensibility evaluation [10 M]

Steps to add an 'Institutional' customer category

- **Update the config, not the code:** add an "Institutional" entry with its own late_fee_percent and slabs list to tariff_config.json — no Python file needs to change for the pricing rules themselves, since slab rates are entirely data-driven.
- **Add one small class:** in consumers.py, add a class InstitutionalConsumer(WaterConsumer) with a compute_bill() method that simply calls the existing self._slab_amount() helper, exactly like the other three subclasses — roughly 2 lines of new code.
- **Register it in one map:** add "Institutional": InstitutionalConsumer to the CATEGORY_CLASS_MAP dictionary in consumers.py so the billing engine can find it automatically.
- **No other file changes:** billing_engine.py, tariff_loader.py, exceptions.py, and the report writers require zero modification, because they already work generically over 'whatever categories exist in the config and the class map' rather than hardcoding category names.

How many files must change, and why: exactly 2 files — tariff_config.json (to add the new category's rates) and consumers.py (to add the new subclass and register it in the map). This is the direct benefit of the Task (a) design: because compute_bill() is abstract and every category's slab logic already lives in the shared _slab_amount() helper, and because the billing engine, validators, and report writers all iterate over CATEGORY_CLASS_MAP / the loaded config rather than checking category names with if/elif chains, adding a new category is purely additive — nothing that already works has to be touched or re-tested.

Two improvements for scaling to 1 million customers

- **Stream and batch instead of loading everything into memory:** the current design reads all rows with csv.DictReader and accumulates every result in a Python list before writing the report, which does not scale to a million rows. The engine should process and write each customer's bill immediately as it is read (or in batches of a few thousand), and use a real database (e.g. PostgreSQL) or a columnar format (e.g. Parquet) instead of a single CSV/TXT pair, so bills_report.txt is replaced by indexed, queryable storage rather than one flat file.
- **Parallelise the billing computation:** since each customer's bill is computed independently of every other customer's, the workload is embarrassingly parallel. Partitioning customers.csv into chunks and processing them concurrently (e.g. with multiprocessing or a distributed framework such as Spark) would let the billing run scale roughly linearly with the number of worker processes/machines, turning a single-threaded run that would take hours on 1 million rows into a job that completes in minutes.

Conclusion

The Smart Water Billing System was implemented as six cooperating pieces: an abstract WaterConsumer base class with three overriding subclasses (Task a), a tariff_loader module that turns tariff_config.json into a structured, validated dictionary (Task b), a billing engine that ties the two together to read customers.csv and apply slab pricing plus a 5% late fee (Task c), layered input validation with four custom exceptions that isolate bad records without stopping the run (Task d), consolidated bills_report.txt and summary_report.txt outputs verified against manual calculation (Task e), and a design that only needs 2 file changes to add a new customer category (Task f). Running the complete system on the sample data billed 10 customers for a combined revenue of Rs.21,540.75 across the three categories, while a separate validation run correctly rejected all three deliberately malformed records with the appropriate custom exception in each case.